

BRANDPULSE: Sprint 3

End-to-End Competitive Intelligence & Machine Learning Campaign Decision Engine

Ismail Rana · Momina

Internship Project · August 2026

BrandPulse — Sprint 1 Recap

Ingested customer conversations from Reddit via RSS feeds

- Ran each post through AI analysis: sentiment + intent classification
- Scored and prioritized posts for the team to act on
- Surfaced results through a live dashboard (FastAPI backend + React frontend)

This was post-level intelligence — every post scored and tagged individually.

Sprint 3: Exploring Narrative-Level Intelligence (Momina's Proposition)

Instead of scoring posts one at a time, group them into narratives — recurring themes and conversations — automatically

- No manual labeling required
- Approach: BGE embeddings turn each post's text into a vector that captures its meaning, then HDBSCAN groups posts with similar vectors into clusters
- Posts that don't clearly belong anywhere are flagged as noise, not forced into a bad-fit group

Limitations

- ~73% of posts didn't fit a clean cluster — reflects real diversity in the Reddit sources, some of which are off-topic (e.g. general tech/hobby subreddits) and need curation
- Used a lightweight embedding model (bge-small) for speed; production would use the larger BGE-M3 for better accuracy and multilingual support
- Clusters aren't persisted yet

Project Evolution: Three Sprints

- ▷ **Sprint 1 — Sentiment & Intent Monitoring:** Tracked Reddit and RSS chatter, but signals were too noisy to justify marketing action.
- ▷ **Sprint 2 — Competitive Vulnerability Engine:** Classified news events and scored opportunities but only produced passive intelligence.
- ▷ **Sprint 3 — ML Campaign Decision Engine:** Bridges detection and execution with a machine-learning model that outputs a ready campaign plan.

Each sprint's limitation directly motivated the next sprint's design.

“

BrandPulse replaces rigid rules with machine learning. It directly converts competitor alerts into active marketing campaigns.

3-Dimensional Decision (Ismail's Proposition)

Sprint 3 automates this decision-making process by predicting a 3-Dimensional Execution Vector:

- **Strategy:** The overarching marketing posture
(COMPETITOR_SWITCHING, PROMOTIONAL_OFFER, RELIABILITY_MESSAGING, PRODUCT_AWARENESS, RETENTION_DEFENSE).
- **Channel:** The optimal distribution vector
(LINKEDIN_EMAIL, SOCIAL_ONLY, WEBINAR_EVENT, CONTENT_MARKETING).
- **Content Type:** The exact collateral format
(COMPARISON_GUIDE, CASE_STUDY, WHITE_PAPER, SOCIAL_CAMPAGN).

Why Machine Learning Over Hard-Coded Rules?

Hard-Coded Rules

Nested if/else thresholds become fragile as more vulnerability types, channels, and content formats are added, and they cannot learn from real campaign outcomes.

Random Forest Model

100 decorrelated decision trees vote on each output, giving probabilistic confidence scores and a model that can be retrained on real conversion data over time.

Model Performance Across Three Outputs

Strategy

84% accuracy

Strongest on
PRODUCT_AWARENESS and
RELIABILITY_MESSAGING.

Channel

88% accuracy

Strongest on LINKEDIN_EMAIL and
CONTENT_MARKETING.

Content Type

89% accuracy

Strongest on WHITE_PAPER and
COMPARISON_GUIDE.



Preprocessing Pipeline

- StandardScaler normalizes severity, urgency, volume, confidence, opportunity score, and article age to zero mean and unit variance.
- OneHotEncoder turns vulnerability_type into orthogonal binary columns, avoiding false ordering between categories.

Solving the Cold-Start Problem

- ▷ **No historical data:** There was zero telemetry linking a competitor vulnerability to a campaign and its outcome.
- ▷ **Weak supervision:** Heuristic mappings and realistic feature distributions (Gaussian severity, exponential article age decay) generated synthetic training examples.
- ▷ **15% noise injection:** Randomizing a slice of labels stopped the model from memorizing rigid thresholds and forced it to learn soft, probabilistic decision boundaries.

Thanks!

Any questions?

BrandPulse — Competitive Intelligence & ML Campaign Decision Engine
Next steps: REST API Frontend & Database implementation